

GrindMate v2.0 — Complete Deep Dive Report

Everything You Need to Know About Your Own Project

What GrindMate Actually Is

GrindMate is NOT a platform to solve problems. It is a **practice management system** that sits on top of LeetCode/NeetCode.

The core loop:

1. You solve a problem on LeetCode or NeetCode
2. You come back to GrindMate and log it in natural language
3. GrindMate tracks your progress, schedules reviews, and reminds you
4. You get an email when reviews are due
5. You review the problem, mark it done, next review scheduled automatically

Why it exists: No other tool does all of this together — LeetCode tracks only LeetCode problems, Notion requires manual effort, nothing sends spaced repetition reminders via email.

File Structure — What Every File Does

```
~/Desktop/GrindMate/
|
├─ src/                                # Backend (Cloudflare Worker)
|   └─ index.ts                        # Main router – entry point for ALL
requests
|   └─ agent.ts                       # Durable Object – the brain of the app
|   └─ auth.ts                        # GitHub OAuth helpers
|   └─ ai.ts                          # Workers AI integration
|   └─ prompts.ts                     # LLM prompts
|   └─ email.ts                       # Resend email + cron logic
|   └─ neetcode.ts                    # NeetCode pattern mapping
|   └─ types.ts                       # TypeScript interfaces
|   └─ data/
|       └─ neetcode150.json           # All 150 NeetCode problems (static)
|
├─ frontend/src/
|   └─ App.tsx                        # Auth gate + sidebar navigation
|   └─ main.tsx                       # React entry point
|   └─ index.css                      # @import "tailwindcss"
|   └─ pages/
|       └─ Dashboard.tsx              # Stats, charts, NeetCode progress
|       └─ Chat.tsx                  # AI chat interface
|       └─ Import.tsx                 # Coming soon page
|
├─ migrations/
|   └─ 0001_add_neetcode.sql           # Added neetcode/neetcode_category columns
|   └─ 0002_add_user_id.sql           # Added user_id, rebuilt composite PKs
|   └─ 0003_add_re_attempt.sql        # Added re_attempt column
|   └─ 0004_add_user_emails.sql       # Added user_emails table
|
├─ schema.sql                         # Full DB schema for fresh installs
├─ wrangler.toml                      # Cloudflare config (bindings, cron, env)
├─ .dev.vars                          # Local secrets (gitignored)
└─ package.json                       # Scripts including all migrations
```

File-by-File Explanation

src/index.ts — The Router

What it does: Every HTTP request hits this file first. It's a Hono router that:

- Serves the React frontend (static assets)

- Handles `/auth/*` routes (GitHub OAuth)
- Routes `/api/*` requests to the correct Durable Object
- Sets `X-User-Id` header on every internal request to DO
- Handles the `/__scheduled` cron trigger
- Handles `scheduled()` for the daily email cron

Key pattern:

typescript

```
// Every API request gets user ID injected
const userId = getCurrentUser(request);
const stub = getUserAgent(env, userId);
// Forward request to that user's Durable Object
```

Why X-User-Id header: `state.id.name` (the DO's own ID) was undefined at runtime — a Cloudflare bug/limitation. The fix was to pass the user ID explicitly via header from `index.ts`.

`src/agent.ts` — The Brain (Durable Object)

What it does: This is the most important file. Each user gets their own instance of this class. It handles:

Method	What It Does
<code>handleLogProblem()</code>	Parse natural language, save to D1, schedule review
<code>handleGetStats()</code>	Return dashboard stats from D1
<code>handleReviewsRequest()</code>	Return due reviews sorted by priority
<code>handleGetNeedsPractice()</code>	Return specific unsolved problems for weak patterns
<code>handleGetNeetCodeProgress()</code>	Return 18-category NeetCode breakdown
<code>scheduleReviews()</code>	Create review #1 (1 day out) in DO storage
<code>markReviewCompleted()</code>	Complete review, schedule next one
<code>alarm()</code>	Fires when DO alarm triggers (keeps DO alive for reviews)
<code>seedGuestData()</code>	Seeds 18 demo problems for guest_demo user

Key design: Reviews live in DO storage (not D1) because D1 has no review columns. Stats queries go to D1. Everything is scoped to `this.userId`.

`src/auth.ts` — GitHub OAuth

What it does:

- `getGitHubAuthURL()` — builds the GitHub redirect URL
- `getGitHubToken()` — exchanges OAuth code for access token
- `getGitHubUser()` — fetches username from GitHub API
- `getGitHubUserEmail()` — fetches primary email (needs `user:email` scope)
- `createSessionToken()` — creates signed base64 session
- `verifySessionToken()` — validates session on each request
- Cookie helpers (set, get, clear)

Important fix made: OAuth scope was `read:user` which doesn't include email access. Changed to `user:email` to allow fetching email for Resend notifications.

`src/ai.ts` — Workers AI Integration

What it does: Three AI tasks using Llama 3.3 70B:

1. `detectIntent(message)` — classifies user message into:
 - LOG_PROBLEM, GET_STATS, GET_RECOMMENDATION, GET_WEEKLY_SUMMARY, GET_REVIEWS, GENERAL
2. `parseProblem(message)` — extracts structured data:

```
json
{
  "leetcode_id": 42,
  "title": "Trapping Rain Water",
  "difficulty": "hard",
  "patterns": ["arrays", "two_pointers"],
  "time_spent_min": 45,
  "struggled": false
}
```

3. `getRecommendations(stats)` — suggests what to practice based on weak patterns

Bug that was fixed: `detectIntent()` had its own stale inline prompt that never listed GET_REVIEWS. This caused "show my reviews" to always fall through to

GENERAL_CHAT. Fixed by adding GET_REVIEWS to the actual prompt and validIntents list.

`src/email.ts` — Email Notifications

What it does:

- `sendDailyReminders(env)` — main cron function:
 1. Gets all distinct user_ids from D1 (excluding guest_demo)
 2. For each user, hits their DO's `/reviews` and `/stats` endpoints
 3. If reviews due → sends email via Resend API
 4. Logs results (sent/skipped/failed)
- `sendReviewEmail()` — POSTs to Resend API
- `emailTemplate()` — HTML email with problem list, streak, CTA button

Why hybrid D1 + DO: Review data only exists in DO storage (scheduled_for, completed). D1 only has the list of users. So cron must query D1 for users, then ask each user's DO for their reviews.

Security: `/__scheduled` endpoint is guarded by `CRON_SECRET` query param. Without it returns 401.

`src/neetcode.ts` — NeetCode Matching

What it does:

- `matchNeetCode(title, leetcodeId)` — checks if a problem is in NeetCode 150
 - Matches by `leetcode_id` first (exact)
 - Falls back to normalized title matching
- `PATTERN_TO_NEETCODE_CATEGORIES` — maps AI pattern tags to NeetCode categories:

```
"two_pointers" → "Two Pointers"
"dynamic_programming" → ["1-D Dynamic Programming", "2-D Dynamic Programming"]
"backtracking" → "Backtracking"
```

Why needed: AI uses free-form tags like "two_pointers" but NeetCode uses "Two Pointers". This map bridges them.

`src/data/neetcode150.json` — Static Problem Data

What it is: All 150 NeetCode problems as JSON. Each entry has:

```
json
{
  "leetcode_id": 1,
  "title": "Two Sum",
  "difficulty": "easy",
  "category": "Arrays & Hashing",
  "slug": "two-sum"
}
```

Source: Pulled from NeetCode's official GitHub repo (`.problemSiteData.json`), filtered to neetcode150 entries. Public data, properly credited.

Why bundled: No API needed, no network request, no rate limiting. Static JSON bundled with the Worker.

`frontend/src/App.tsx` — Main App

What it does:

- Checks auth status on load (`/api/me`)
 - Shows login page if not authenticated (GitHub OAuth + Guest Demo button)
 - Shows sidebar navigation if authenticated
 - Shows demo banner if `user === 'guest_demo'`
 - Routes between Dashboard, Chat, Import pages
-

`frontend/src/pages/Dashboard.tsx` — The Dashboard

What it shows:

1. **Stats cards:** Total Solved, Current Streak, Best Streak, Patterns, Reviews Due
2. **By Difficulty:** Donut chart (Easy/Medium/Hard breakdown)
3. **Top Patterns:** Bar chart of most practiced patterns
4. **NeetCode 150 Progress:** Overall progress bar + 18 category grid
5. **Needs Practice:** Weak patterns with specific unsolved NeetCode problems + LeetCode links
6. **Reviews Modal:** Clickable from Reviews Due card — shows due problems with  /  priority

Bug fixed: `{stats?.reviews_due && ...}` rendered "0" when reviews_due was 0 (number, not boolean). Fixed to `{stats && stats.reviews_due !== undefined && stats.reviews_due > 0 && ...}`.

Database Schema

sql

```
-- Problems you've solved
problems (
  id, user_id, leetcode_id, title, difficulty,
  patterns (JSON), time_spent_min, struggled,
  neetcode (boolean), neetcode_category,
  re_attempt (boolean), solved_at
)

-- For streak calculation
daily_activity (
  user_id, date (composite PK),
  problems_solved, total_time_min
)

-- Pattern mastery
pattern_progress (
  user_id, pattern (composite PK),
  solved_count, last_practiced
)

-- Email notifications
user_emails (
  user_id (PK), email,
  notify_enabled, created_at
)
```

Key design decision: `(daily_activity)` and `(pattern_progress)` use composite primary keys `(user_id, date)` and `(user_id, pattern)`. Without this, two users solving on the same day would overwrite each other's rows via ON CONFLICT upserts — a critical multi-tenant bug.

Spaced Repetition System

How it works:

Day 0: You log "solved two sum easy 10 min"
→ Review #1 scheduled for Day 1

Day 1: Reviews Due count goes up on dashboard
→ "show my reviews" → "Two Sum (Easy) – solved 1 day ago 🟡"
→ You solve it on LeetCode, come back
→ "reviewed two sum"
→ ✅ Review #1 done! Review #2 scheduled for Day 3 (3 days from now)

Day 4: Review #2 due
→ Same flow
→ Review #3 scheduled for Day 7 (7 days from now)

Day 11: Review #3 done → "Two Sum mastered! 🎉 No more reviews."

Priority ordering:

- 🟡 Problems marked "struggled" (showed difficulty)
- 🟢 Normal problems

Where reviews live: Durable Object storage (not D1). Each ReviewItem has:

```
typescript
{
  id: string,           // unique ID for completion
  leetcode_id: number,
  title: string,
  difficulty: string,
  patterns: string[],
  solved_on: string,    // when originally solved
  review_number: number, // 1, 2, or 3
  scheduled_for: string, // ISO date
  completed: boolean,
  struggled: boolean    // for priority ordering
}
```

Due-ness computed live: Never cached. Always `(scheduled_for <= today && !completed)`. Can't drift from truth even if alarm doesn't fire.

Authentication Flow

1. User clicks "Login with GitHub"
2. GET /auth/login → redirect to GitHub
3. User approves on GitHub
4. GitHub → GET /auth/callback?code=xxx
5. Exchange code for access token (server-side)
6. Fetch GitHub username + primary email
7. Save email to user_emails table
8. Create signed session cookie
9. Redirect to frontend
10. Every subsequent request → verify cookie → extract user_id → route to DO

Session: Base64 signed token in httpOnly cookie. Verified on every request. **Email capture:** Happens at OAuth callback. GitHub token discarded after — never stored.

Guest Demo Mode

How it works:

- "View Demo" button → GET /auth/guest → sets session with `user_id = 'guest_demo'`
- On first request to guest DO, seeds 18 problems if empty:
 - 8 easy, 8 medium, 2 hard
 - 7-day streak history
 - 3 NeetCode categories in progress
 - 2 struggled + 1 normal review due (shows    priority)
- All write operations blocked (LOG_PROBLEM, markReview, re-attempt)
- Guest sees dedup message for seeded problems (correct behavior)
- Guest excluded from email cron

Why: Recruiters can try the app instantly without GitHub account. Demo banner shows at top.

Email System

Stack: Resend API + Cloudflare cron trigger

Schedule: Every day at 8 AM UTC (`0 8 * * *`)

Flow:

1. Cron fires → `scheduled()` handler in index.ts
2. `sendDailyReminders()` in email.ts

3. `SELECT DISTINCT user_id FROM problems` (excludes `guest_demo`)
4. For each user → hit their DO's /reviews endpoint
5. If reviews due → send email via Resend
6. Log: `(X sent, Y skipped, Z failed)`

Manual trigger: `(GET /__scheduled?secret=YOUR_CRON_SECRET)`

From address: `(onboarding@resend.dev)` (Resend test address, no domain needed)

Security: `CRON_SECRET` env var guards the manual trigger endpoint

What Makes It Unique (Trademarks)

1. **Natural language logging** — no forms, no dropdowns, just type
 2. **Auto-review scheduling** — every problem gets reviewed, not just "struggled" ones
 3. **Email reminders** — works even when you don't open the app
 4. **Per-user Durable Objects** — true isolation, not just `WHERE user_id` filtering
 5. **NeetCode 150 integration** — roadmap + specific problem recommendations
 6. **Guest demo** — recruiters can try it instantly
 7. **Edge-native** — sub-50ms globally, no cold start pain
-

⚠ Known Limitations

Limitation	Why	Workaround
LeetCode import blocked	LeetCode blocks cloud IPs (403)	Manual logging via chat
AI can't verify problem names	LLM infers patterns, not verified ground truth	Cross-reference against neetcode150.json planned
state.id.name undefined	Cloudflare runtime bug	X-User-Id header workaround
No timezone support	Streaks use server UTC	User's local midnight may differ
Single Resend domain	Using test address (onboarding@resend.dev)	Need custom domain for production
Guest is shared identity	guest_demo is one DO instance for all guests	Read-only enforced, ephemeral chat
AI quota	~100-200 calls/day on free tier	Enough for personal use, upgrade for scale
Reviews only in DO storage	Can't query all users' reviews in one D1 query	Hybrid D1 + DO approach in cron
No re-attempt reviews	Re-attempt doesn't create new review chain	By design — original review chain continues

? Interview Questions & Answers

Architecture

Q: Why Cloudflare over AWS?

"Integrated stack — Workers, Durable Objects, D1, Workers AI all on the same edge network. No latency between services. Single deployment command. Free tier covers my usage. Trade-off is vendor lock-in."

Q: What's a Durable Object?

"A stateful actor at the edge. Each user gets their own DO instance — like a tiny server dedicated to that user. It persists state between requests, supports scheduled alarms, and runs single-threaded so no race conditions. Not a database, not a cache — it's closer to an actor model."

Q: Why not just use D1 for everything?

"D1 is great for structured queries. But review queues and chat history need fast read/write without cold queries. DOs keep state in memory between requests. Also DOs support alarms (scheduled wake-ups) which power spaced repetition without an external cron service."

Q: How does user isolation work?

"Two layers. First, each user gets their own Durable Object instance — keyed by GitHub username. Second, D1 tables use composite primary keys (user_id, date) and (user_id, pattern) so upserts can't overwrite other users' rows. I learned this the hard way — initially I only had DO isolation, but D1 upserts were still colliding between users."

AI

Q: How does natural language parsing work?

"Two-stage pipeline. Stage 1: classify intent (LOG_PROBLEM, GET_STATS, etc.) with a focused prompt and few-shot examples. Stage 2: if LOG_PROBLEM, run a specialized extraction prompt to get structured JSON. This keeps each prompt focused and accurate."

Q: What if the AI returns bad JSON?

"Validation at every step. Strip markdown backticks, regex-extract the JSON object, parse it, check required fields exist, validate enums. If anything fails, return a friendly error asking the user to rephrase — never crash or silently save bad data."

Q: How do you handle AI hallucinations in pattern extraction?

"Currently I trust the LLM's inference — it knows Two Sum uses arrays and hash maps from training data. That's a known limitation. The right fix is cross-referencing against a verified problem-to-pattern dataset. It's on the roadmap."

Database

Q: Why patterns stored as JSON array in D1?

"I considered a junction table (problem_patterns) but the query patterns don't justify it. I rarely need 'all problems with X pattern' — I mostly need 'all patterns for this problem.' JSON array is simpler and SQLite's JSON functions handle the rare cross-pattern query."

Q: How do you prevent duplicate problems?

"Before inserting, check if leetcode_id OR normalized title already exists for this user. If yes, prompt for re-attempt confirmation. Re-attempts insert with re_attempt=TRUE"

and don't start a new review chain. Stats count only `re_attempt=FALSE` rows."

Q: How does streak calculation work?

"Query `daily_activity` for consecutive days with `problems_solved > 0`, ordered by date DESC. Count backwards until a gap. Known limitation: uses server UTC, not user's local timezone — a user solving at 11 PM EST might miss their streak if the day rolls over."

Bugs & Debugging

Q: What was the hardest bug?

"The user isolation bug. All users were seeing my data. I thought Durable Objects would handle isolation since each user gets their own DO instance. But D1 uses upserts with `ON CONFLICT` — without composite primary keys, two users solving on the same date would overwrite each other's `daily_activity` row. The fix was rebuilding `daily_activity` and `pattern_progress` with composite `(user_id, date)` and `(user_id, pattern)` primary keys."

Q: Tell me about the `state.id.name` bug.

"I was getting the user ID from `state.id.name` inside the Durable Object — that's the name passed to `idFromName()` when creating the DO. At runtime this was undefined even though it should work per docs. Cloudflare runtime bug or undocumented behavior. Fix was passing the user ID explicitly via `X-User-Id` header from `index.ts`, set from the session cookie."

Q: How did you find the reviews bug?

"Reviews always showed 0. I traced the flow — chat said 'show my reviews' → AI classified it as `GENERAL` not `GET_REVIEWS`. Found that `detectIntent()` had its own stale inline prompt that never listed `GET_REVIEWS` as a valid intent. Dead code in `prompts.ts` had the right prompt but was never used. One-line fix, hour of debugging."

Security

Q: How is the app secured?

"Several layers: GitHub OAuth for authentication, `httpOnly` signed session cookies (prevents XSS), composite primary keys for data isolation, `CRON_SECRET` for the scheduled endpoint, guest mode is fully read-only (403 on write attempts), HTML-escaping in email templates to prevent injection from malicious problem titles."

Q: What would you improve security-wise?

"Rate limiting on the API endpoints — currently anyone could spam the chat endpoint. The `/__scheduled` endpoint has `CRON_SECRET` but could also have IP allowlist."

Session expiry could be shorter. And I should add CSRF protection on state-changing endpoints."

Trade-offs

Q: What trade-offs did you make?

"Three main ones:

1. Cloudflare native stack — great DX and free tier, but vendor lock-in. Can't migrate easily.
2. Reviews in DO storage not D1 — fast access but can't query across users. The email cron has to hit each user's DO individually.
3. LLM for pattern extraction — flexible input, but not verified ground truth. A wrong pattern tag silently corrupts progress tracking."

Q: What would you do differently?

"Add user_id to schema from day one instead of discovering the isolation bug later. Skip LeetCode import — researching API limitations before building would have saved hours. Write integration tests for multi-user scenarios early — would have caught the isolation bug immediately."

Metrics to Quote

Metric	Value
Problems in NeetCode 150 JSON	150
DSA categories tracked	18
Spaced repetition intervals	1, 3, 7 days
Review chain length	3 reviews → mastered
Email cron schedule	8 AM UTC daily
D1 migrations shipped	4
Cold start time	sub-50ms (Cloudflare edge)
Free tier limit	100K requests/day

How to Run It

Local development:

bash

Terminal 1 – Backend

```
cd ~/Desktop/GrindMate  
npx wrangler dev --remote
```

Terminal 2 – Frontend

```
cd ~/Desktop/GrindMate/frontend  
npm run dev
```

Deploy:

bash

```
cd ~/Desktop/GrindMate/frontend && npm run build && cd .. && npx wrangler deploy
```

Trigger email cron manually:

bash

```
curl "https://grindmate.sanketjanger15.workers.dev/__scheduled?secret=YOUR_CRON_SECRET"
```

Check live logs:

bash

```
npx wrangler tail
```

Built by Sanket Janger / MS CS @ Binghamton University Stack: TypeScript · React · Hono · Cloudflare Workers · Durable Objects · D1 · Workers AI · Resend · GitHub OAuth